

Lab # 10

Linux Shell Scripting

I. Introduction to Shell Programming

A shell script is a text file containing a list of commands to be executed by the Linux shell. Running a shell script is equivalent to typing the commands within it at the shell prompt. Shell program is a series of Linux commands. Useful to create our own commands that can save our lots of time and to automate some tasks.

II. Your First Shell Script

Step 1: Creating a script file

\$ vi first.sh or vi first

Step 2: Specify the shell path

The first line of a script can be used to specify which shell interprets the script. This is done with a #! (Pound sign followed by an exclamation point) followed by the absolute pathname to the shells executable name. The BASH shell's executable name is Bash and the absolute pathname on most Linux systems is /bin/bash. The first line of the script should be

#!/bin/bash

Step 3: Write the script.

Any text following a pound sign (#) is considered a comment and is ignored. Using the #! Sign as in Step 2 is an exception to this rule. For example, add the following lines to your script.

```
#!/bin/bash

#

# My first script

#
clear

echo "Knowledge is Power"
```

Step 4: Make the script file executable.

Use the chmod command to make the script file executable. For example,

\$ chmod 755 first.sh

This tells Linux that the script file should be treated as a Linux command.

Step 5: Run the script.

To run script type;

\$./first.sh

III. Variables in Linux

Variables are one of the programming features that make shell scripting so powerful. Sometimes to process our data/information, it must be kept in computers RAM memory. RAM memory is divided into small locations, and each location had unique number called memory location/address, which is used to hold our data. Programmer can give a unique name to this memory location/address called memory variable or variable (It's a named storage location that may take different values, but only one at a time). In Linux, there are two types of variable

a) System variables

Created and maintained by Linux itself. This type of variable defined in CAPITAL LETTERS.

b) User defined variables (UDV)

Created and maintained by user. This type of variable defined in lower LETTERS.

Example:

```
$ echo  
$USERNAME  
$ echo  
$HOME
```

How to define User defined variables (UDV)

To define UDV use following syntax

Syntax:

```
variablename = value
```

NOTE: Here 'value' is assigned to given 'variablename' and Value must be on right side = sign. For Example

```
$ no=10  
$ 10=no  
# Error To define variable called "vech" having value Bus  
$ vech=Bus  
To define variable called n having value 20  
$ n=20
```

Note: Don't put spaces on either side of the equal sig when assigning value to variable. Variables are case-sensitive, just like filename in Linux.

```
$ echo  
$ no          # will print 10  
$ echo  
$ vech        # will print Bus  
$ echo  
$ n           # will print 20
```

You can define NULL variable as follows (NULL variable is variable which has no value at the time of definition).

```
$vech=
$vech=""
```

Try to print its value \$ echo \$vech, Here nothing will be shown because variable has no value i.e. NULL variable.

IV. Shell Arithmetic

Use to perform arithmetic operations.

Syntax: `expr op1 math-operator op2`

Examples:

```
$ expr 1 + 3
```

```
$ expr 2 - 1
```

```
$ expr 10 / 2
```

```
$ expr 20 % 3          # remainder read as 20 mod 3 and remainder is 2
```

```
$ expr 10 \* 3         # Multiplication use \* not * since its wild card
```

```
$ echo `expr 6 + 3`
```

For the last statement note the following points

1. First, before `expr` keyword we used ``` (back quote) sign not the (single quote i.e. `'`) sign. Back quote is generally found on the key under tilde (~) on PC keyboard OR to the above of TAB key.
2. Second, `expr` is also end with ``` i.e. back quote.
3. Here `expr 6 + 3` is evaluated to 9, then `echo` command prints 9 as sum.
4. Here if you use double quote or single quote, it will NOT work.

```
$ echo "expr 6 + 3"    # It will print expr 6 + 3
```

```
$ echo 'expr 6 + 3'    # It will work
```

More about Quotes

There are three types of quotes

Quotes	Name	Meaning
"	Double Quotes	"Double Quotes" - Anything enclosed in double quotes removed meaning of that characters (except \ and \$).
'	Single quotes	'Single quotes' - Enclosed in single quotes remains unchanged.
`	Back quote	`Back quote` - To execute command.

Example:

```
$ echo "Today is date"
```

Can't print message with today's date.

```
$ echo "Today is `date`".
```

Now it will print today's date as, Today is Tue Jan, Note that the `date` statement uses back quote.

V. The “read” Statement

Use to get input (data from user) from keyboard and store (data) to variable.

Syntax: read variable1, variable2, ... variableN

```
$ vigetin
```

```
#!/bin/bash
```

```
#Script to read your Name from keyboard
```

```
#
```

```
Echo “Enter Your First Name:”
```

```
read name
```

```
echo “Hello $name”
```

To get input in same line use `-n` option with echo command.

Like; echo `-n` “Enter Your First Name:”

LAB TASK

- 1.Take two inputs from user and then perform arithmetic operations
- 2.Use system variables and user defined variables to show how that value can be accessed.
- 3.Show results of double , single and back quote.